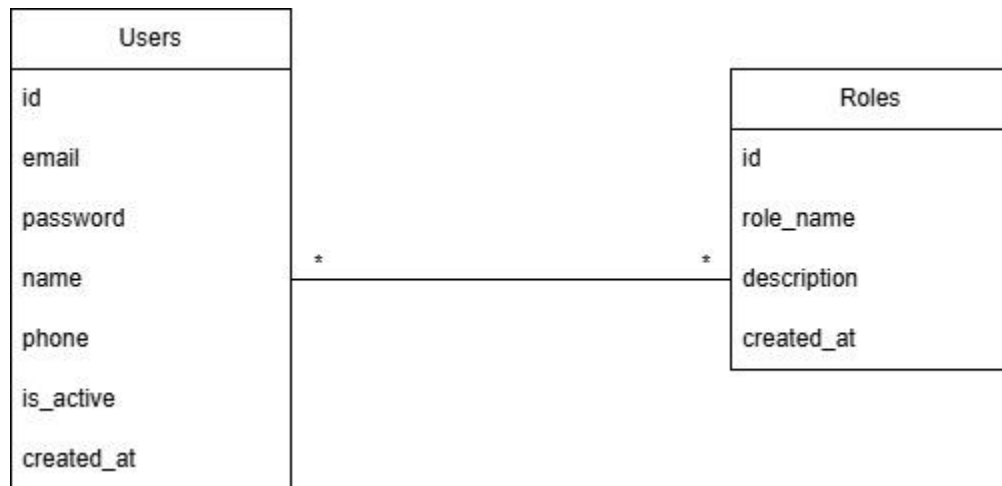


PREPARCIAL - SISTEMA DE USUARIOS Y ROLES

OBJETIVO GENERAL

Crear una aplicación con NestJS que maneje:

- Autenticación con JWT usando Passport.
- Autorización por roles con decoradores y guards personalizados.
- Persistencia en PostgreSQL con relación muchos a muchos entre usuarios y roles.
- Endpoints protegidos según permisos.



REQUERIMIENTOS FUNCIONALES

MODELOS:

Tabla users:

- `id` (PK, uuid)
- `email` (string, único, not null)
- `password` (string, hash, not null)
- `name` (string)
- `phone` (string, opcional)
- `is_active` (boolean, default true)
- `created_at` (timestamp, default now)

Tabla roles:

- `id` (PK, uuid)
- `role_name` (string, único)

- description (string, opcional)

ENDPOINTS OBLIGATORIOS:

Para cada endpoint indico: URI, método, códigos de éxito, ejemplos de errores mínimos, rol/permiso requerido y body si es necesario.

Registro

- URI: POST /auth/register
- Descripción: crear usuario (el rol puede ser asignado directamente en este endpoint o puede ser asignado más adelante)
- Success:
 - 201 Created → { message: "Usuario registrado con éxito", userId: <id> }
- Errores mínimos:
 - 400 Bad Request → "Email inválido"
 - 409 Conflict → "Email ya registrado"
- Acceso: público

Login

- URI: POST /auth/login
- Descripción: recibe email + password; retorna access_token
- Success:
 - 200 OK → { access_token: "<jwt>" }
- Errores:
 - 401 Unauthorized → "Credenciales incorrectas"
 - 423 Locked → "Usuario desactivado" (si is_active=false)
- Acceso: público

Crear rol

- URI: POST /roles
- Descripción: crear nuevo rol
- Success:
 - 201 Created → { message: "Rol creado con éxito", roleId: <id> }
- Errores:
 - 400 Bad Request → "role_name es requerido"
 - 401 Unauthorized → "Token inválido o faltante"
 - 403 Forbidden → "No autorizado"
 - 409 Conflict → "role_name ya existe"
- Acceso: admin

Listar roles

- URI: GET /roles
- Success:
 - 200 OK → [{ id, role_name, description }, ...]
- Errores:
 - 401 Unauthorized → "Token inválido o faltante"
 - 403 Forbidden → "No autorizado"
 - 500 Internal Server Error → "Error al obtener roles"
- Acceso: admin

Asignar roles a usuario

- URI: PATCH /users/:id/roles
- Body: { roles: ["admin","doctor"] }
- Success:
 - 200 OK → { message: "Roles asignados" }
- Errores:
 - 400 Bad Request → "roles inválidos"
 - 401 Unauthorized → "Token inválido o faltante"
 - 403 Forbidden → "No autorizado"
 - 404 Not Found → "Usuario no encontrado"
- Acceso: admin

Obtener perfil propio

- URI: GET /users/me
- Success:
 - 200 OK → { id, email, name, phone, roles: [...] }
- Errores:
 - 401 Unauthorized → "Token inválido o faltante"
 - 404 Not Found → "Usuario no encontrado"
- Acceso: autenticado

Listar usuarios

- URI: GET /users
- Success:
 - 200 OK → [{ id, email, name, roles: [...] }, ...]
- Errores:
 - 401 Unauthorized → "Token inválido o faltante"
 - 403 Forbidden → "No autorizado"
 - 500 Internal Server Error → "Error al listar usuarios"
- Acceso: admin

REQUERIMIENTOS TÉCNICOS

Autenticación y autorización

- Usar **Passport** con estrategia passport-jwt.
- Hashear passwords con **bcrypt** (bcryptjs o bcrypt).
- Configuración completa de JWT y variables de entorno:
 - JWT_SECRET
 - JWT_EXPIRES_IN ('120s')
- Token en Authorization: Bearer <token>.

Decorador y Guard

- Implementar @Roles(...roles: string[]) que guarde metadata.
- Implementar RolesGuard que:
 - Obtiene usuario del request (payload de JWT).
 - Compara con roles requeridos y retorna 403 si no coincide.

Persistencia y migraciones

- Usar **TypeORM** y entregar migraciones SQL.
- Instrucciones para conectar a PostgreSQL vía variables de entorno:
 - DB_HOST
 - DB_PORT
 - DB_USER
 - DB_PASS
 - DB_NAME

Validaciones y seguridad

- Validar entradas con class-validator y DTOs.
- Nunca devolver el campo password en respuestas.

ENTREGABLES

1. Migraciones SQL de la BD
2. Script SQL de usuarios y roles probados
3. Release de repositorio en Github a más tardar jueves 14 de mayo del 2026 a las 5 PM